

SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

• **Theme:** Theme 01 — Agentic Code Intelligence

Team: VIT_CodeNexus_1

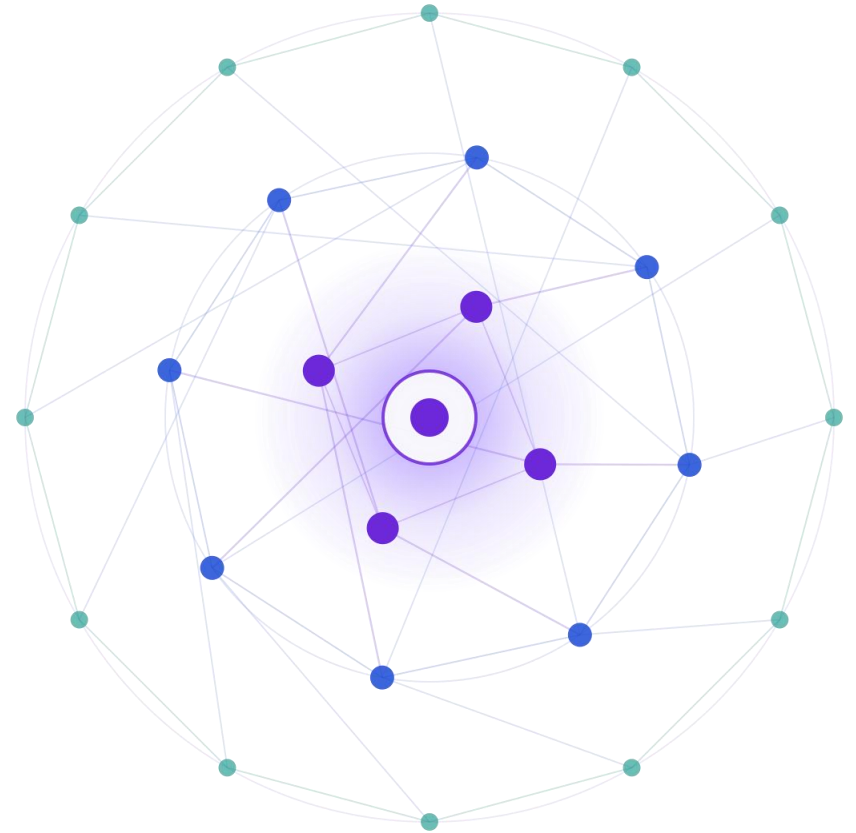
College: Vellore Institute of Technology

Member 1: Mithun Venkatesan (mithun.venkatesan2023@vitstudent.ac.in)

Member 2: Kavin M (kavin.m2023@vitstudent.ac.in)

Member 3: Akhiil Dheep B (akhiildheep.b2023@vitstudent.ac.in)

GitHub: https://github.com/Mithun-Newt/agentic_code_intelligence.git



Theme: Agentic Code Intelligence

The problem

- AI coding agents must find the right files and code blocks before they can fix bugs or add features.
- This is code retrieval: given a natural-language query, rank code snippets by relevance.
- It is hard because queries describe intent, while code is terse and shares few keywords with the query.

Scope of our submission

- **Shipped (P0):** dense semantic retrieval, run on CPU.
- **Out of scope:** LLM answer generation, code explanation, chatbots.
- **Future work:** P1/P2 items on the “What’s Next” slide; none are implemented.

Screening benchmark: CoIR AppsRetrieval (MTEB) test split

3,765 test queries · 8,765 code documents · Metrics: NDCG@10, MRR@10

Existing Solutions & Gaps

Experiment 0 baseline

Flat dense retrieval with intfloat/e5-base-v2, default “query: ” / “passage: ” prefixes, and exact cosine search over normalized vectors.

Measured on the full CoIR Apps test split.

0.11523

NDCG@10

0.09878

MRR@10

0.16919

Recall@10

Gaps in existing approaches

1

Semantic-to-code divide

Queries describe an abstract problem; the matching code has terse names and little keyword overlap.

2

General-purpose embeddings

Standard text embeddings are not tuned for code retrieval. The E5 baseline reaches only 0.11523 NDCG@10.

3

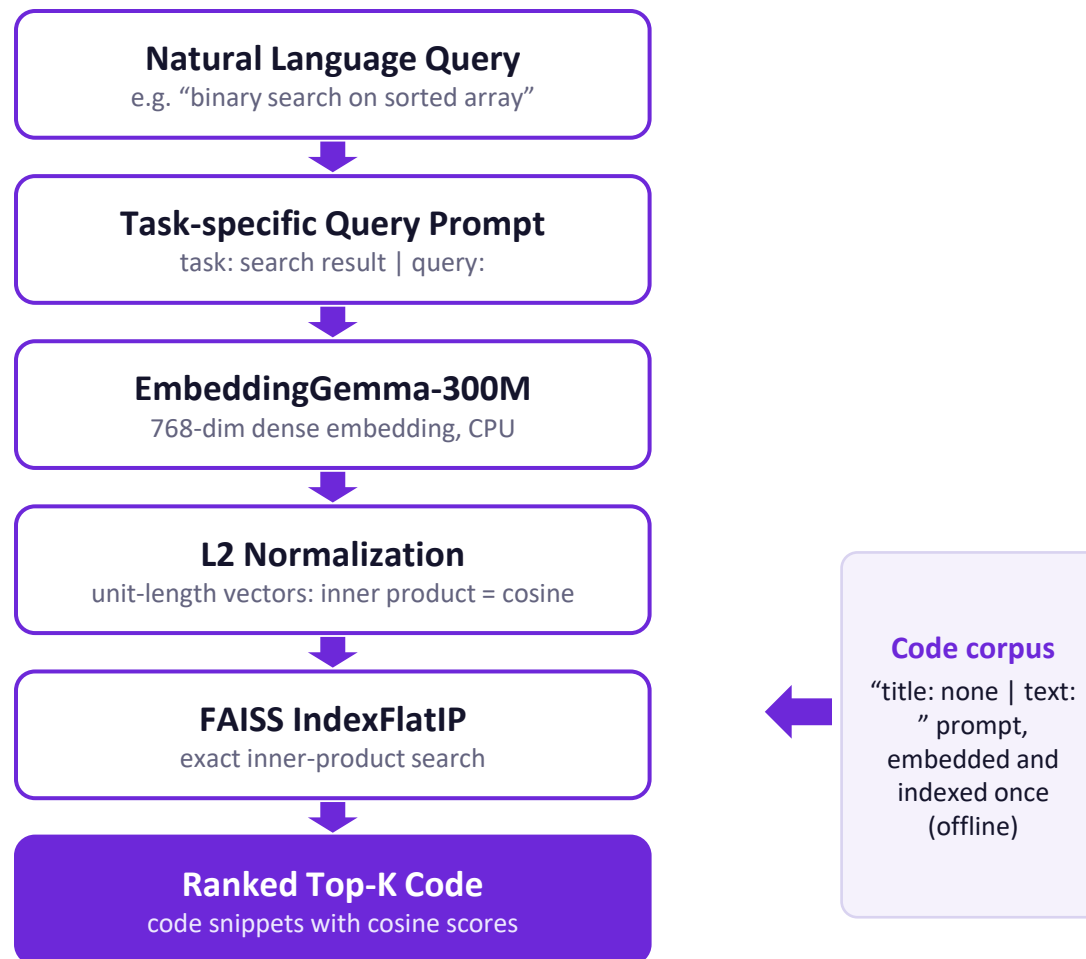
Heavy ranking pipelines

LLM-based ranking is slow and cannot fit thousands of long snippets in context; the brief expects CPU-friendly retrieval.

Our Solution & Architecture

Our method: P0 dense retrieval

- **Model:** google/embeddinggemma-300m (300M parameters, 768-dim), chosen by a controlled comparison on 200 validation queries.
- **Task-specific prompts:** queries use “task: search result | query: ”; code uses “title: none | text: ”.
- **Exact search:** L2-normalized embeddings with FAISS IndexFlatIP, so inner product equals cosine similarity.
- **Integration:** implemented as an MTEB AbsEncoder (PrePostPipelineEncoder).
- **Access fallback:** RedHatAI/embeddinggemma-300m mirror if the original model cannot be downloaded.



Demo & Product Walkthrough

Command-line demo (demo.py)

- **Input:** a natural-language query.
- **Output:** ranked snippets with rank, cosine score, document ID, title and code.
- **Corpus:** 15 curated algorithms (searching, sorting, graph traversal, dynamic programming).
- **Runs locally on CPU;** no cloud API calls once model weights are cached.
- **Latency:** search time is printed; about 90–140 ms per query on this corpus.

```
$ python demo.py --query "binary search on sorted array" --top-k 3

Loaded 15 code documents. Indexing corpus...
[Search Completed in 94.62 ms on CPU]

[Rank 1] Score: 0.6318 | doc_binary_search
Title: Binary Search on Sorted Array (Algorithms/Searching)
def binary_search(arr: list[int], target: int) -> int:
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2 ... return mid

[Rank 2] Score: 0.3120 | doc_quick_select
[Rank 3] Score: 0.2845 | doc_merge_sort
```

Output on the 15-document demo corpus, CPU only.

Tools & Tech Stack

Embedding inference

- sentence-transformers 6.1.0: model wrapper and batch embedding
- transformers 4.57.6 and torch 2.14.0 (CPU), 10 intra-op threads

Evaluation

- mteb 2.21.6: official harness for the AppsRetrieval task
- datasets 5.0.1: loads the CoIR benchmark test split

Vector indexing

- faiss-cpu 1.15.1: exact IndexFlatIP search over L2-normalized vectors
- numpy 2.2.6: array handling and metric computation

Quality & deployment

- pytest 9.1.1: 9 automated tests, all passing
- Docker (python:3.10-slim): multi-stage container for reproducibility

Impact & Use Case

Use case

- Code retrieval is the first step for AI coding assistants: locating relevant code from a plain-language description.
- Fits IDE assistants and code-review tools.

CPU-friendly

- Runs on CPU with no cloud API calls at query time.
- Demo query latency is about 90–140 ms on the 15-document corpus.

Easy to integrate

- Follows the MTEB AbsEncoder interface.
- Modular pipeline: prompt formatting, embedding, then indexing.

The shipped system is the retrieval component only; it does not generate answers or plan multi-step agent actions.

Innovation, Results & Limitations

Innovation highlights

- Controlled model comparison on 200 validation queries before the full test run.
- Task-specific query and code prompts.
- Model-access fallback for reliable runs.

Limitations (P0 scope)

- Dense retrieval only: no answer generation or multi-step agent reasoning.
- 512-token cap truncates long source files.
- Pure dense retrieval can miss rare identifier names.

Measured official test results

MTEB AppsRetrieval, CoIR Apps test split (3,765 queries, 8,765 documents)

Metric	Experiment 0 baseline (E5-base-v2)	Final P0 engine (EmbeddingGemma-300M)	Gain
NDCG@10	0.11523	0.80492	+598.5% (7.0×)
MRR@10	0.09878	0.77097	+680.5% (7.8×)
Recall@10	0.16919	0.91049	+438.1% (5.4×)
Recall@100	0.34343	0.98114	+185.7% (2.9×)
Recall@1000	0.68127	0.99655	+46.3%

Baseline and final engine were evaluated on the same full test split. The full CPU test evaluation took 112 minutes.

What's Next: Future Work

Future Work only: none of these items is implemented in the shipped P0 system.

FUTURE WORK · P1

AST-aware code chunking

Split source files at function, method and class boundaries (e.g. with Tree-sitter) instead of fixed token cuts, so long files are not truncated.

FUTURE WORK · P1

Hybrid sparse + dense retrieval

Combine BM25 keyword matching with dense embeddings using Reciprocal Rank Fusion (RRF) to catch rare identifier names.

FUTURE WORK · P1

Multi-version repository indexing

Re-index only the files changed in each Git commit instead of rebuilding the whole index.

FUTURE WORK · P2

Agentic multi-hop retrieval

Refine the query iteratively based on intermediate results.

Differentiation (Brownie Points)

Verified accuracy

- NDCG@10 rose from 0.11523 (Experiment 0) to 0.80492, a gain of +0.68969.
- Measured on the full CoIR Apps test split.

Transparent scope

- Every metric comes from a real CPU run.
- Shipped system is strictly P0 dense retrieval; the roadmap is labelled Future Work.

Reproducible and tested

- CLI demo on a fixed 15-document corpus.
- 9/9 automated tests passing; Docker definition included.

Focused on P0

- All effort went into retrieval accuracy, the top-priority goal in the Theme 01 brief.

Checklist: Updated on Public GitHub

YES

Working prototype code (public GitHub repo)

https://github.com/Mithun-Newt/agentic_code_intelligence.git

YES

README with reproducible setup instructions

Installation, CLI demo and MTEB evaluation commands

YES

Demo video (max 5 minutes)

https://drive.google.com/file/d/1ygrzH9wNctm0r5rT4pi3kkJrUA_bPuJz/view?usp=sharing

YES

Presentation file (PPT or PDF)

submission/CodeNexus_PRISM_Theme01_Final.pptx and .pdf

Thank you

CodeNexus · Vellore Institute of Technology

https://github.com/Mithun-Newt/agentic_code_intelligence.git

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India